In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [4]:
```
#import dataset
df=pd.read_csv("train.csv")
```

In [5]:
```
df.head()
```

Out[5]:

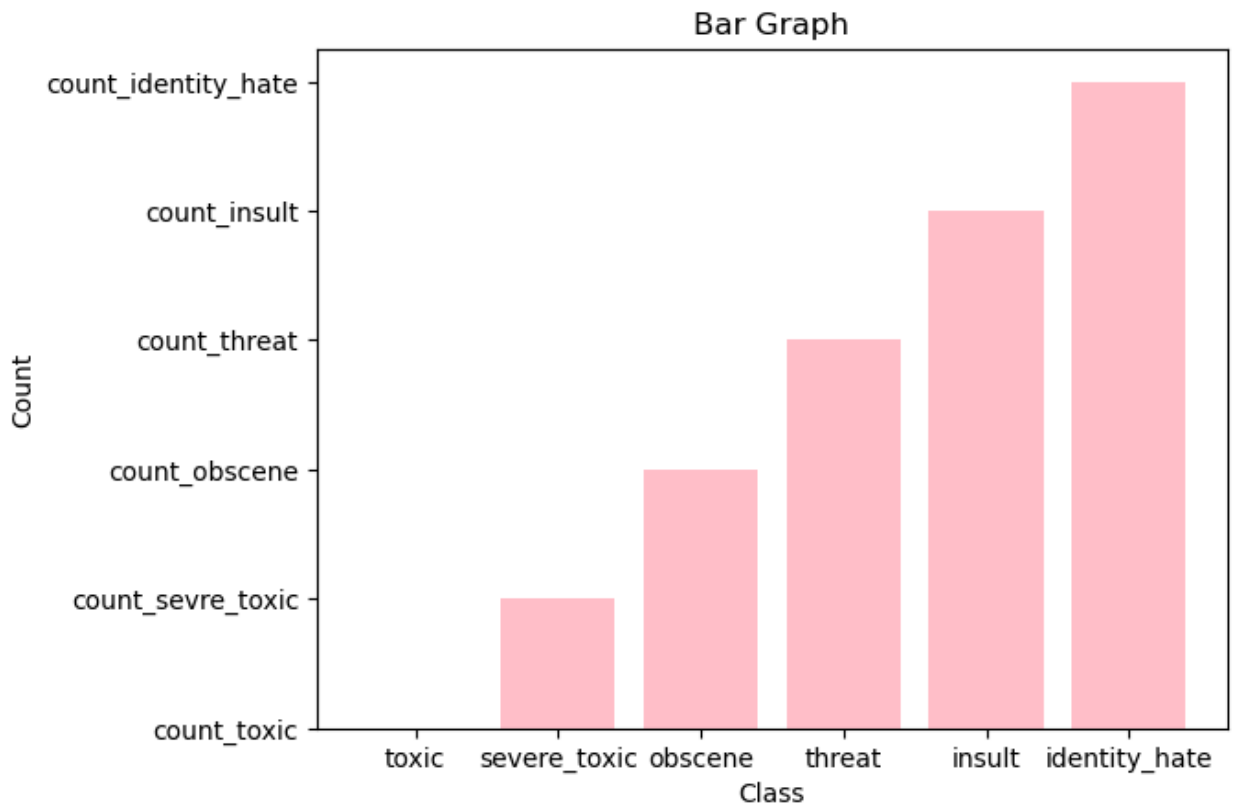
	id	comment_text	toxic	severe_toxic	obscene	threat	insult
0	0000997932d777bf	Explanation\nWhy the edits made under my usern...	0	0	0	0	0
1	000103f0d9cfb60f	D'aww! He matches this background colour I'm s...	0	0	0	0	0
2	000113f07ec002fd	Hey man, I'm really not trying to edit war. It...	0	0	0	0	0
3	0001b41b1c6bb37e	"\nMore\nI can't make any real suggestions on ...	0	0	0	0	0
4	0001d958c54c6e35	You, sir, are my hero. Any chance you remember...	0	0	0	0	0

In [13]:
```
Class = ['toxic', 'severe_toxic','obscene','threat','insult','identity_hate']
c=df[Class].sum()
print(c)
```

```
toxic            15294
severe_toxic      1595
obscene           8449
threat             478
insult            7877
identity_hate     1405
dtype: int64
```

In [11]:
```
count_toxic=df['toxic'].value_counts()
count_sevre_toxic=df['severe_toxic'].value_counts()
count_obscene=df['obscene'].value_counts()
count_threat=df['threat'].value_counts()
count_insult=df['insult'].value_counts()
count_identity_hate=df['identity_hate'].value_counts()
Class = ['toxic', 'severe_toxic','obscene','threat','insult','identity_hate']
count=['count_toxic','count_sevre_toxic','count_obscene','count_threat','count
plt.bar(Class,count,color='pink')
plt.xlabel("Class")
```

```
plt.ylabel("Count")
plt.title("Bar Graph")
plt.show()
print(count_toxic,count_sevre_toxic,count_obscene,count_threat,count_insult,cc
```




```
toxic
0    144277
1     15294
Name: count, dtype: int64 severe_toxic
0    157976
1      1595
Name: count, dtype: int64 obscene
0    151122
1      8449
Name: count, dtype: int64 threat
0    159093
1       478
Name: count, dtype: int64 insult
0    151694
1      7877
Name: count, dtype: int64 identity_hate
0    158166
1      1405
Name: count, dtype: int64
```

In [14]:
```
df['MERGED_HATE_SPEECH'] = df['severe_toxic'] + df['obscene']+df['toxic']+df['

df['MERGED_HATE_SPEECH'] = (( df['severe_toxic']>0)|(df['obscene']>0) |(df['tc
).astype(int)
```

```
print(df)
```

```
                     id                                       comment_text  \
0       0000997932d777bf  Explanation\nWhy the edits made under my usern...
1       000103f0d9cfb60f  D'aww! He matches this background colour I'm s...
2       000113f07ec002fd  Hey man, I'm really not trying to edit war. It...
3       0001b41b1c6bb37e  "\nMore\nI can't make any real suggestions on ...
4       0001d958c54c6e35  You, sir, are my hero. Any chance you remember...
...                  ...                                                ...
159566  ffe987279560d7ff  ":::::And for the second time of asking, when ...
159567  ffea4adeee384e90  You should be ashamed of yourself \n\nThat is ...
159568  ffee36eab5c267c9  Spitzer \n\nUmm, theres no actual article for ...
159569  fff125370e4aaaf3  And it looks like it was actually you who put ...
159570  fff46fc426af1f9a  "\nAnd ... I really don't think you understand...

        toxic  severe_toxic  obscene  threat  insult  identity_hate  \
0           0             0        0       0       0              0
1           0             0        0       0       0              0
2           0             0        0       0       0              0
3           0             0        0       0       0              0
4           0             0        0       0       0              0
...       ...           ...      ...     ...     ...            ...
159566      0             0        0       0       0              0
159567      0             0        0       0       0              0
159568      0             0        0       0       0              0
159569      0             0        0       0       0              0
159570      0             0        0       0       0              0

        MERGED_HATE_SPEECH
0                        0
1                        0
2                        0
3                        0
4                        0
...                    ...
159566                   0
159567                   0
159568                   0
159569                   0
159570                   0

[159571 rows x 9 columns]
```

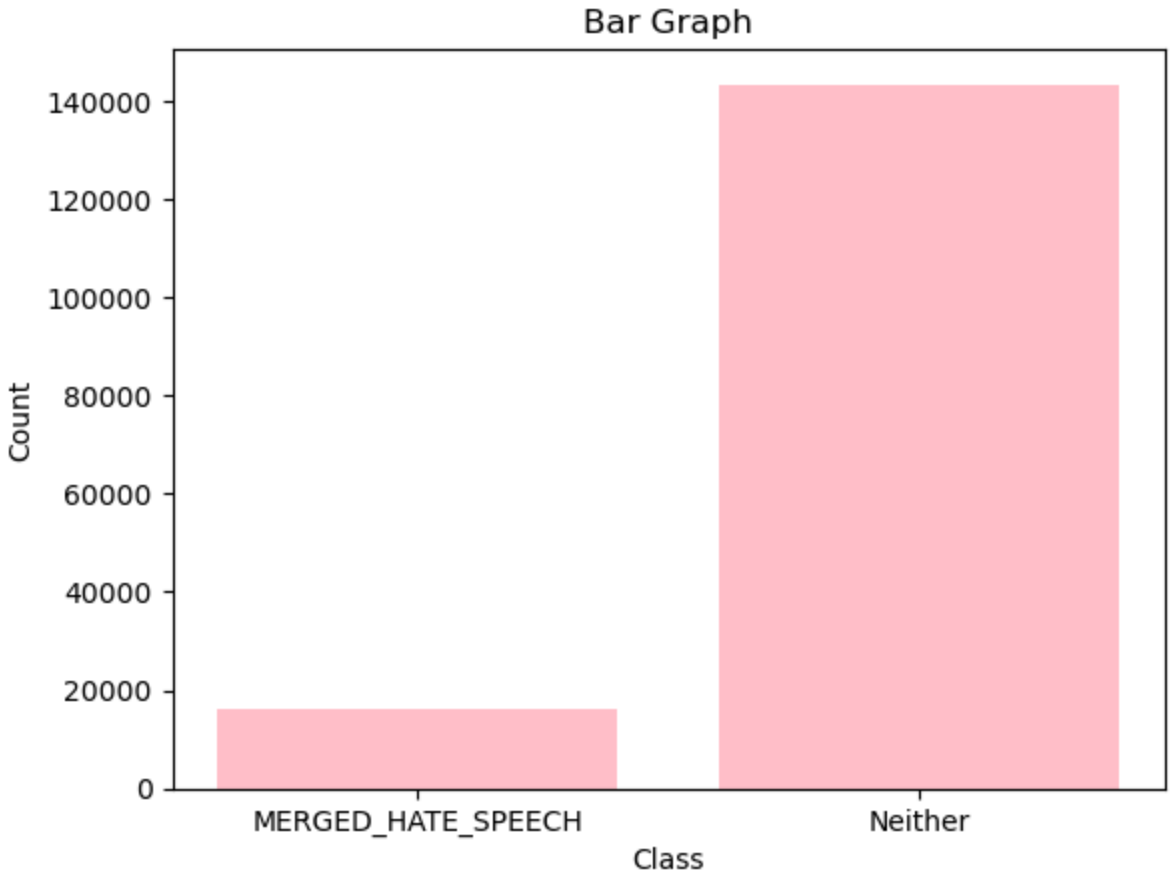
In [15]:
```
df['Neither'] = (df['MERGED_HATE_SPEECH'] == 0).astype(int)
print(df)
```

```
                     id                                       comment_text  \
0       0000997932d777bf  Explanation\nWhy the edits made under my usern...
1       000103f0d9cfb60f  D'aww! He matches this background colour I'm s...
2       000113f07ec002fd  Hey man, I'm really not trying to edit war. It...
3       0001b41b1c6bb37e  "\nMore\nI can't make any real suggestions on ...
4       0001d958c54c6e35  You, sir, are my hero. Any chance you remember...
...                  ...                                                ...
159566  ffe987279560d7ff  ":::::And for the second time of asking, when ...
159567  ffea4adeee384e90  You should be ashamed of yourself \n\nThat is ...
159568  ffee36eab5c267c9  Spitzer \n\nUmm, theres no actual article for ...
159569  fff125370e4aaaf3  And it looks like it was actually you who put ...
159570  fff46fc426af1f9a  "\nAnd ... I really don't think you understand...

        toxic  severe_toxic  obscene  threat  insult  identity_hate  \
0           0             0        0       0       0              0
1           0             0        0       0       0              0
2           0             0        0       0       0              0
3           0             0        0       0       0              0
4           0             0        0       0       0              0
...       ...           ...      ...     ...     ...            ...
159566      0             0        0       0       0              0
159567      0             0        0       0       0              0
159568      0             0        0       0       0              0
159569      0             0        0       0       0              0
159570      0             0        0       0       0              0

        MERGED_HATE_SPEECH  Neither
0                        0        1
1                        0        1
2                        0        1
3                        0        1
4                        0        1
...                    ...      ...
159566                   0        1
159567                   0        1
159568                   0        1
159569                   0        1
159570                   0        1

[159571 rows x 10 columns]
```

In [21]:
```
c=['MERGED_HATE_SPEECH','Neither']
cout=df[c].sum()
print(cout)
```

```
MERGED_HATE_SPEECH     16225
Neither               143346
dtype: int64
```

In [22]:
```
plt.bar(c,cout,color='pink')
plt.xlabel("Class")
plt.ylabel("Count")
plt.title("Bar Graph")
plt.show()
```


Bar Graph
140000
120000
100000
80000
60000
40000
20000
0
Count
MERGED_HATE_SPEECH
Neither
Class

In [ ]:

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
```

In [2]:
```
df=pd.read_csv("KNN_Cancer_Data.csv")
```

In [3]:
```
df
```

Out[3]:

	diagnosis	radius_mean	texture_mean
0	M	17.99	10.38
1	M	20.57	17.77
2	M	19.69	21.25
3	M	11.42	20.38
4	M	20.29	14.34
...	...	...	...
564	M	21.56	22.39
565	M	20.13	28.25
566	M	16.60	28.08
567	M	20.60	29.33
568	B	7.76	24.54

569 rows × 3 columns

In [4]:
```
X = df[['radius_mean', 'texture_mean']]
Y=df['diagnosis']
```

In [5]:
```
X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.2)
```

In [6]:
```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, Y_train)
```

Out[6]:

KNeighborsClassifier

Parameters

n_neighbors	5
weights	'uniform'
algorithm	'auto'
leaf_size	30
p	2
metric	'minkowski'
metric_params	None
n_jobs	None

In [7]:
```
y_pred = knn.predict(X_test)
```

In [8]:
```
from sklearn.metrics import accuracy_score,confusion_matrix,classification_rep
```

In [9]:
```
#ACCURACY
a=accuracy_score(y_pred,Y_test)
print(f"Accuracy of proposed model:{a}")
```

```
Accuracy of proposed model:0.868421052631579
```

In [10]:
```
#CONFUSSION_MATRIX
cm=confusion_matrix(y_pred,Y_test)
print(f"Confusion matrix is:-\n{cm}")
```

```
Confusion matrix is:-
[[76  7]
 [ 8 23]]
```

In [11]:
```
#CLASSIFICATION_REPORT
Output=['Malignant','Begnin']
cr=classification_report(y_pred,Y_test,target_names=Output)
print(cr)
```

```
              precision    recall  f1-score   support

   Malignant       0.90      0.92      0.91        83
      Begnin       0.77      0.74      0.75        31

    accuracy                           0.87       114
   macro avg       0.84      0.83      0.83       114
weighted avg       0.87      0.87      0.87       114
```

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df=pd.read_csv("Housing.csv")
```

In [3]:
```
df.head()
```

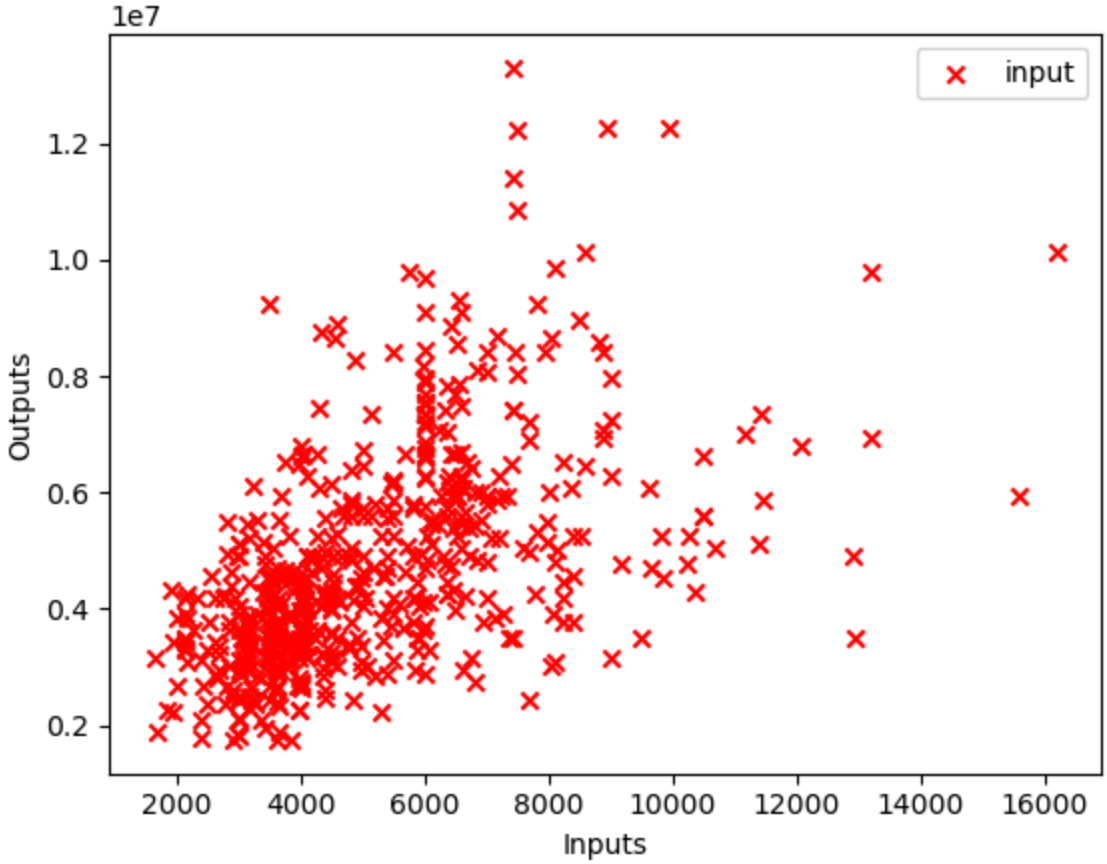
Out[3]:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basem
0	13300000	7420	4	2	3	yes	no	
1	12250000	8960	4	4	4	yes	no	
2	12250000	9960	3	2	2	yes	no	
3	12215000	7500	4	2	2	yes	no	
4	11410000	7420	4	1	2	yes	yes	

In [4]:
```
X=df['area']
Y=df['price']
```

In [5]:
```
import matplotlib.pyplot as plt
```

In [6]:
```
#SCATTER PLOT
plt.scatter(X,Y,marker="x",c='r',label="input")
plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.legend()
plt.show()
```



In [7]:
```
# Linear regression model => y=wx+b
#def linear_regression_model(X,w,b):
 #   pred_y=np.zeros(len(X))
  #  for i in range (len(X)):
   #     pred_y[i]=w*X[i]+b
    #return pred_y
```

In [12]:
```
w=1200
b=10
alpha=0
```

In [13]:
```
# Linear regression model => y=wx+b
def linear_regression_model(X,w,b):
    pred_y=np.zeros(len(X))
    for i in range (len(X)):
        pred_y[i]=w*X[i]+b
    return pred_y
```

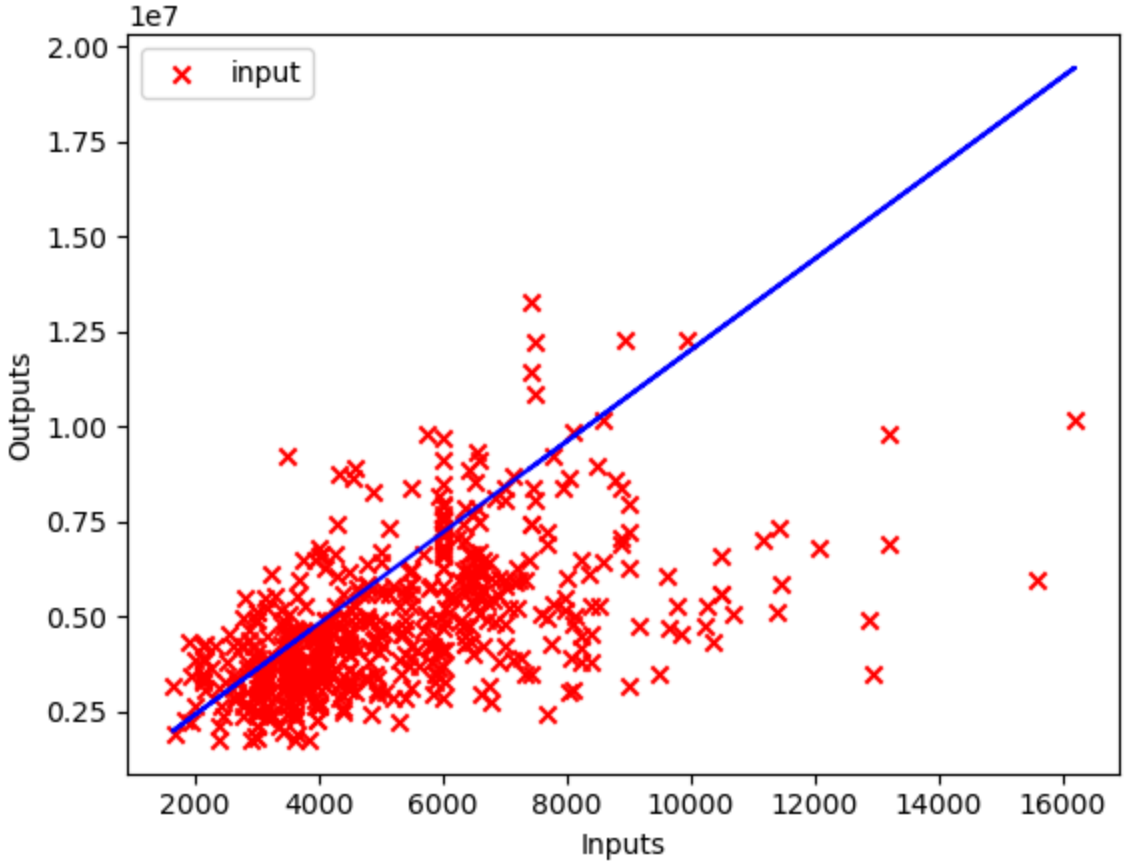
In [14]:
```
pred_y=linear_regression_model(X,w,b)
print(pred_y)
```

```
[ 8904010. 10752010. 11952010.  9000010.  8904010.  9000010. 10296010.
 19440010.  9720010.  6900010. 15840010.  7200010.  7860010.  4200010.
  9360010.  7200010.  7920010. 10200010.  5520010.  7704010.  5184010.
  8586010.  9660010.  5472010. 10560010.  7848010.  7200010. 10650010.
  9540010.  6600010.  8970010.  8400010.  5856010.  7152010.  8208010.
  8400010.  8978410. 10800010.  7200010.  7200010.  7860010.  7632010.
  7776010.  7200010.  7200010.  7200010.  7200010.  7920010.  5160010.
  8928010.  8928010.  7590010.  7200010.  6180010.  7200010.  7200010.
 13728010. 10800010.  9216010.  7200010.  7200010. 10656010.  7488010.
  7632010. 13410010. 10656010. 15840010.  9240010.  7200010. 14508010.
  4800010.  7200010.  6024010.  7920010.  4848010.  5112010.  7704010.
  7800010.  6840010.  7200010.  7200010.  4800010. 12600010.  7200010.
  4512010.  9900010.  8004010.  4752010.  8892010. 10296010.  6000010.
  8100010.  5760010.  8640010.  7200010.  4920010. 10800010.  7680010.
  7920010.  7200010.  7920010.  6600010.  6600010.  7620010.  6600010.
  5400010.  6540010.  7704010.  3888010.  7938010.  7920010. 10046410.
  5160010. 11544010.  8160010.  9600010.  8280010.  4440010.  7704010.
  8424010.  7848010.  8677210.  7504810.  8784010.  7830010. 18720010.
  8592010.  7800010.  6600010. 13752010.  5760010.  6993610.  6240010.
  5760010.  8400010.  7200010.  6480010.  5568010.  6000010.  7632010.
  6960010.  7992010. 12600010.  5760010.  5640010.  6000010. 12600010.
  6600010.  7632010.  7920010.  6163210.  5280010.  6480010.  3960010.
  4380010.  7320010.  8280010.  3380410.  9576010.  3780010.  7452010.
  7320010.  7920010.  8190010.  8052010.  7740010.  9360010.  5520010.
  5112010.  7848010.  6600010. 12322810. 10080010.  6360010.  4560010.
 11760010. 10224010.  7260010.  8502010.  3816010.  5400010.  8640010.
  4092010.  9576010.  3600010.  3600010. 13692010.  7320010.  6864010.
  4248010.  9120010. 12840010.  7920010.  5760010.  9780010.  5292010.
  9223210.  3360010.  7137610.  5040010.  5424010.  4914010.  4944010.
  6480010.  5724010.  7560010.  6960010.  3600010.  3564010.  8064010.
  5575210. 15480010.  4104010.  5994010.  5220010.  4992010.  7248010.
  8234410.  5778010.  8400010.  9720010.  4104010. 10999210.  7585210.
 12288010.  7728010.  6204010.  7200010.  4356010. 11600410.  6480010.
  5184010.  4494010.  4992010.  4656010.  6816010.  3444010.  6012010.
  5412010.  4800010.  4608010.  4512010.  4368010.  3060010.  6384010.
  6432010.  4224010. 10080010.  4920010.  5988010.  4212010.  4140010.
 11832010.  4224010.  5412010.  7062010.  4800010.  9900010.  4848010.
  7632010.  3794410.  4212010.  4500010.  4761610.  5880010.  3456010.
  5856010.  5904010.  5940010.  4680010.  5400010.  2286010.  4890010.
  4200010.  7740010.  4838410.  5280010. 12432010.  4080010.  7632010.
  7632010.  5400010.  2610010.  5232010.  9324010.  7980010.  3344410.
  6600010.  6048010.  7020010.  3132010.  3543610.  3296410.  5292010.
  4800010.  2790010.  5520010.  4368010.  6960010.  8400010.  4894810.
  4224010.  2574010.  5400010.  9900010.  4140010.  5808010.  4896010.
  4855210.  5558410.  7182010.  7272010.  4320010.  4416010.  4848010.
  6720010.  7080010.  5990410.  5208010.  3600010.  5184010.  4356010.
  4152010.  6480010.  5400010.  4152010.  4920010.  7776010.  5400010.
  4752010.  4860010.  8712010.  6600010.  3600010.  3948010.  4579210.
  9696010.  2574010.  4536010.  3816010.  6360010.  3816010.  8582410.
  4896010.  4620010.  2418010.  2611210.  4020010.  3780010.  5784010.
  4104010.  4320010.  6996010.  3427210. 10080010.  9900010.  3024010.
  8316010.  4176010.  4320010.  4848010.  7224010.  4860010.  4300810.
  3744010.  6540010.  4356010.  4356010.  6768010.  4320010.  5136010.
  4284010.  3816010.  3600010.  4224010.  7152010.  4956010.  3420010.
```

```
2730010.  4224010.  5400010.  4800010.  3780010.  5400010.  5400010.
4368010.  4620010.  5088010.  4380010.  5520010.  2562010.  3643210.
4788010.  8908810.  4176010.  4320010.  4368010.  7080010.  3744010.
8820010.  4214410. 11400010.  7056010. 15532810.  5880010.  3672010.
6384010.  2574010.  4800010.  3822010.  4620010.  2574010.  3132010.
2340010.  4848010.  5742010.  4140010.  4368010.  4200010.  5952010.
4944010.  5700010.  4464010.  4500010.  3720010.  3822010.  3240010.
2574010.  4848010.  5730010.  3000010.  3816010.  7272010.  4176010.
4550410.  4848010.  2574010.  7056010.  5400010.  4716010.  4368010.
5244010.  3220810.  5184010.  3744010.  4140010.  4783210.  4200010.
4914010.  1980010.  4140010.  8100010. 10800010.  3682810.  5400010.
6594010.  2877610.  3600010.  4620010.  4200010.  9720010.  5952010.
2592010.  3708010.  5400010.  4560010.  3708010.  3888010.  3402010.
5520010.  6091210.  4500010.  4356010.  9660010.  5222410.  3600010.
7020010.  5952010.  4320010.  4392010.  4176010.  3240010.  3780010.
7938010.  3648010.  4356010.  7200010.  6480010.  6240010.  3960010.
5220010.  3168010.  3180010.  4752010.  8160010.  4800010.  4800010.
4720810.  2400010.  4356010.  3360010.  2916010.  4176010.  4800010.
3822010.  4800010.  3492010.  4320010.  5280010.  4320010.  3456010.
3816010.  3600010.  5280010.  3600010.  3852010.  3888010.  3600010.
4200010.  5808010.  9240010.  4362010.  2970010.  3344410.  3916810.
4368010.  3816010.  2203210.  4764010.  4764010.  2340010.  6360010.
3600010.  2880010.  3600010.  4032010.  4104010.  2040010.  4378810.
3588010.  3600010.  2880010.  4344010.  3492010.  4620010.]
```

In [15]:

```python
# Plot Linear Regression
plt.scatter(X,Y,marker="x",c='r',label="input")
plt.plot(X,pred_y,c='b')
plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.legend()
plt.show()
```



COST FUNCTION

In [16]:
```
def cost_function_linear_regression(pred_y,Y):
    sum_cost=0
    for i in range(len(X)):
       # pred_y=w*X[i]+b
        sum_cost=sum_cost+(pred_y[i]-Y[i])**2 + alpha*(w**2)
    cost_total=(1/2*len(X))*sum_cost
    return cost_total
```

In [17]:
```
cost=cost_function_linear_regression(pred_y,Y)
print(cost)
```

```
1.0467956316059218e+18
```

In [22]:
```
def gradient_descent_cost_function(X,Y,w,b):
    j_dw, j_db=0,0
    for i in range(len(X)):
        f_x=w*X[i]+b
        j_dw=j_dw + (f_x - Y[i])*X[i] + alpha*w
        j_db=j_db + (f_x - Y[i])

    j_dw=j_dw/len(X)
    j_db=j_db/len(X)
```

```
    return j_dw,j_db
```

In [23]:
```
j_dw,j_db= gradient_descent_cost_function(X,Y,w,b)
print(f"j_dw={j_dw} and j_db={j_db}")
```

```
j_dw=10751866983.394495 and j_db=1413930.2935779816
```

In [24]:
```
def gradient_descent_algorithm(X,Y,w,b,learning_rate,gradient_descent_cost_fun
    w,b=w,b
    for i in range(iteration):
        j_dw,j_db= gradient_descent_cost_function(X,Y,w,b)
        w=w-learning_rate*j_dw
        b=b-learning_rate*j_db
    return w,b
```

In [29]:
```
w=0
b=0
learning_rate=0.001
iteration=10
aplha = 100
w,b=gradient_descent_algorithm(X,Y,w,b,learning_rate,gradient_descent_cost_fun
print(f"w: {w} and b: {b}")
```

```
w: -7.546766694807557e+47 and b: -1.2446764380854083e+44
```

In [ ]:
```
#w=[o1,o2,o3]
#b=[]
#aplha = [0.01,0.1,1]
w=0
b=0
learning_rate=0.001
iteration=10
aplha = 100
w,b=gradient_descent_algorithm(X,Y,w,b,learning_rate,gradient_descent_cost_fun
print(f"w: {w} and b: {b}")
```

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df=pd.read_csv("Brain_Tumor (4).csv")
```

In [3]:
```
df
```

Out[3]:

	Class	Tumor_Size	Unnamed: 2
0	0	98.613971	NaN
1	0	63.858816	NaN
2	0	81.867206	s
3	1	151.229741	NaN
4	1	174.988756	NaN
...	...	...	...
3743	1	158.437600	NaN
3744	1	161.158675	NaN
3745	1	167.130118	NaN
3746	1	223.812932	NaN
3747	1	239.251388	NaN

3748 rows × 3 columns

In [4]:
```
y=df['Class']
x=df[['Tumor_Size']]
```

In [5]:
```
#2d array
#x=np.array(x).reshape(-1,1)
#y=np.array(y).reshape(-1,1)
```

In [ ]:

In [6]:
```
x
```

Out[6]:

	Tumor_Size
0	98.613971
1	63.858816
2	81.867206
3	151.229741
4	174.988756
...	...
3743	158.437600
3744	161.158675
3745	167.130118
3746	223.812932
3747	239.251388

3748 rows × 1 columns

In [7]:
```
y
```

Out[7]:
```
0       0
1       0
2       0
3       1
4       1
       ..
3743    1
3744    1
3745    1
3746    1
3747    1
Name: Class, Length: 3748, dtype: int64
```

In [8]:
```
print(f"Total Inputs={len(x)}")
print(f"Total Outputs={len(y)}")
```

```
Total Inputs=3748
Total Outputs=3748
```

In [9]:
```
print(df['Class'].value_counts())
```

```
Class
0    2155
1    1593
Name: count, dtype: int64
```

In [10]:
```
#SPILT THE DATASET USING TRAIN_TEST_SPLIT() METHOD
from sklearn.model_selection import train_test_split
```

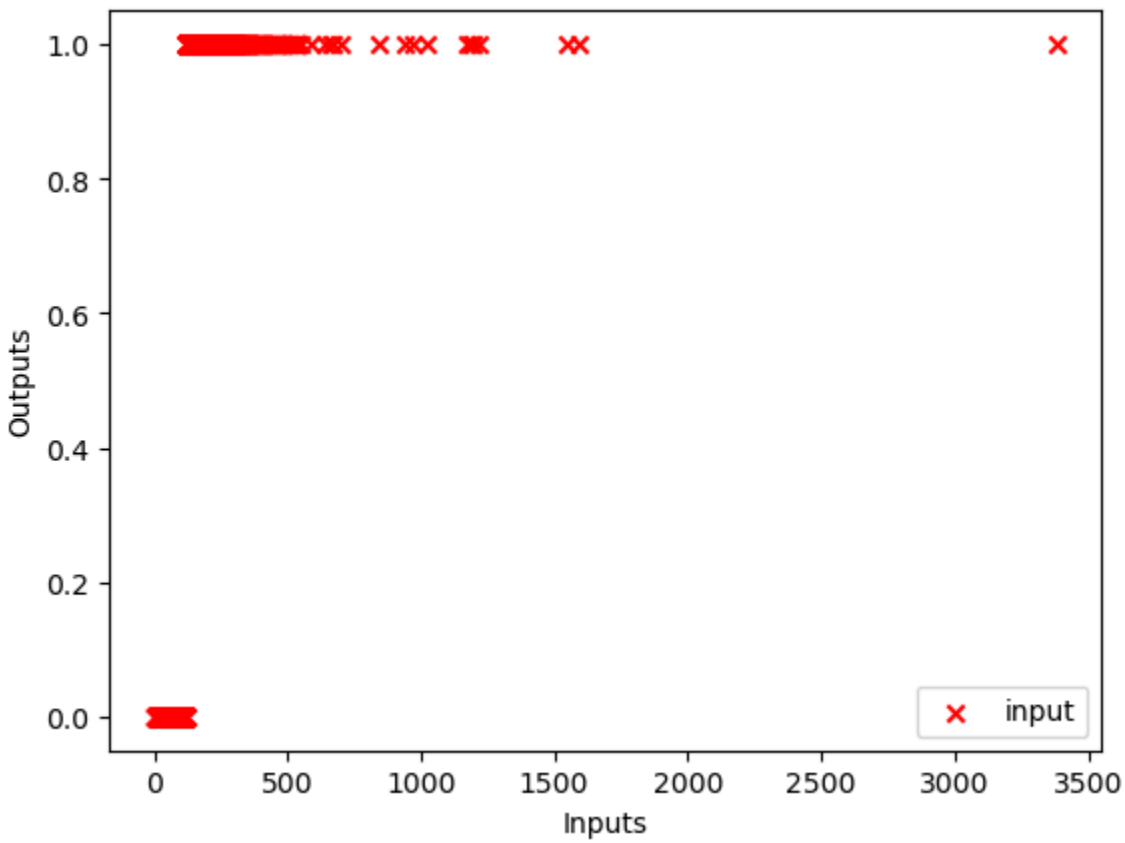
In [11]:
```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2)
```

In [12]:
```
print(f"Total Training Inputs={len(x_train)}")
print(f"Total Testing Outputs={len(x_test)}")
```

```
Total Training Inputs=2998
Total Testing Outputs=750
```

In [13]:
```
import matplotlib.pyplot as plt
```

In [14]:
```
#SCATTER PLOT
plt.scatter(x,y,marker="x",c='r',label="input")
plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.legend()
plt.show()
```



In [15]:
```
#Make Logistic Regression Model using sklearn
from sklearn.linear_model import LogisticRegression
```

In [16]:
```
model_log=LogisticRegression()
```

In [17]:
```
#fit the model/train the model
model_log.fit(x_train,y_train)
```

Out[17]:

LogisticRegression

Parameters

penalty	'l2'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	100
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

In [19]:

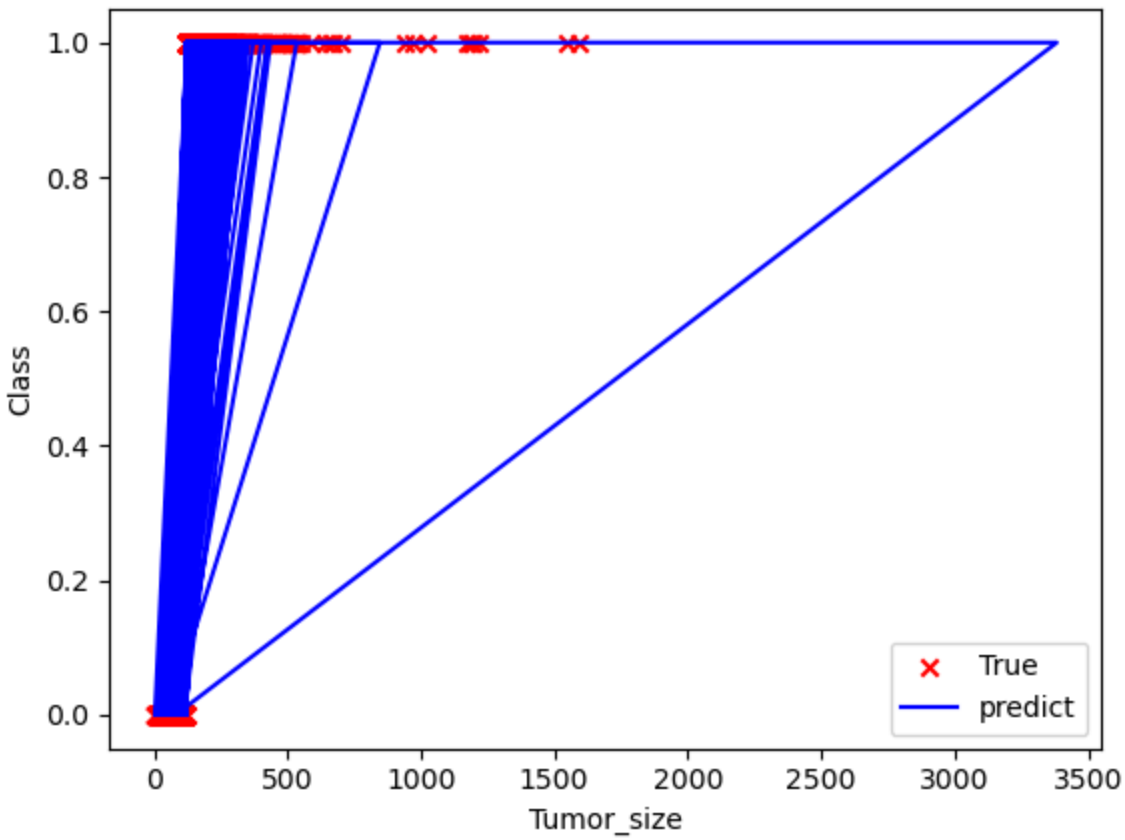
```
#prediction of model
y_pred=model_log.predict(x_test)
```

In [20]:

```
y_pred
```

Out[20]:
```
array([1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0,
       1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1,
       0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0,
       0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0,
       0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0,
       0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0,
       1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0,
       1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0,
       0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0,
       1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1,
       0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0,
       0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1,
       0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0,
       0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1,
       0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0,
       1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0,
       1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1,
       1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0,
       0, 0])
```

In [21]:
```python
#SCATTER PLOT
plt.scatter(x_train,y_train,marker="x",c='r',label="True")
plt.plot(x_test,y_pred,c='b',label="predict")
plt.xlabel('Tumor_size')
plt.ylabel('Class')
plt.legend()
plt.show()
```



In [22]:
```
from sklearn.metrics import accuracy_score,confusion_matrix,classification_rep
```

In [25]:
```
#ACCURACY
a=accuracy_score(y_pred,y_test)
print(f"Accuracy of proposed model:{a}")
```

```
Accuracy of proposed model:1.0
```

In [28]:
```
#CONFUSSION_MATRIX
cm=confusion_matrix(y_pred,y_test)
print(f"Confusion matrix is:-\n{cm}")
```

```
Confusion matrix is:-
[[449   0]
 [  0 301]]
```

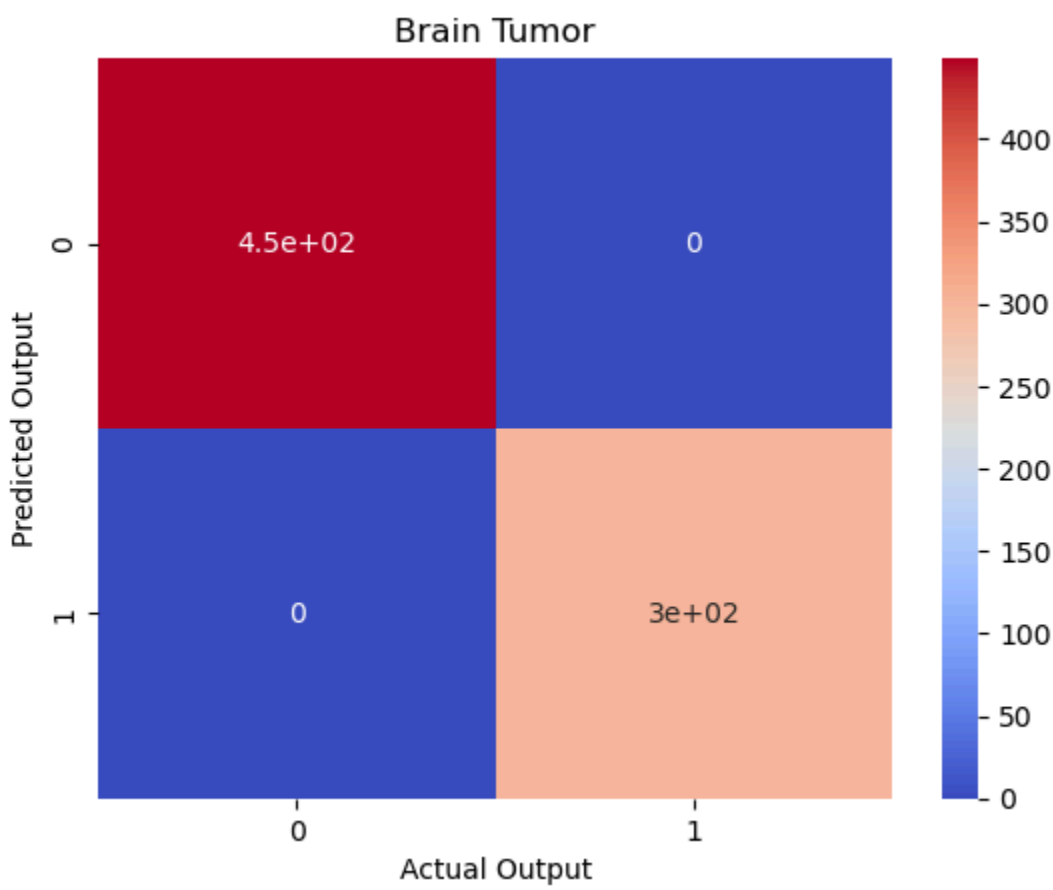
In [30]:
```
#CLASSIFICATION_REPORT
Output=['Malignant','Begnin']
cr=classification_report(y_pred,y_test,target_names=Output)
print(cr)
```

```
              precision    recall  f1-score   support

   Malignant       1.00      1.00      1.00       449
      Begnin       1.00      1.00      1.00       301

    accuracy                           1.00       750
   macro avg       1.00      1.00      1.00       750
weighted avg       1.00      1.00      1.00       750
```

In [31]:
```
import seaborn as sns
```

In [33]:
```
sns.heatmap(cm,cmap='coolwarm',annot=True,xticklabels=[0,1],yticklabels=[0,1])
plt.xlabel("Actual Output")
plt.ylabel("Predicted Output")
plt.title("Brain Tumor")
```

Out[33]: Text(0.5, 1.0, 'Brain Tumor')



In [ ]:

In [44]:
```
import numpy as np
```

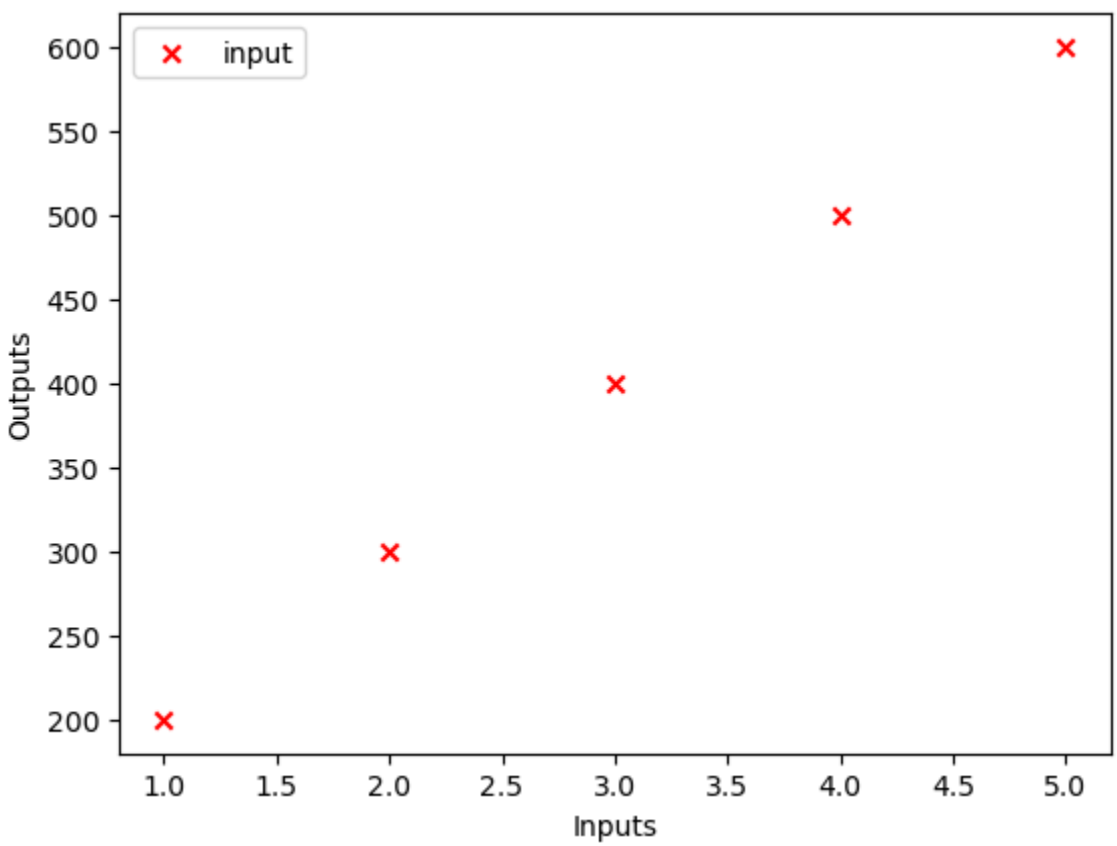
In [45]:
```
X=np.array([1,2,3,4,5])
Y=np.array([200,300,400,500,600])
```

In [46]:
```
#Display All Training Examples
for i in range(len(X)):
    print(f"(X[{i}],Y[{i}])=({X[i]},{Y[i]})")
```

```
(X[0],Y[0])=(1,200)
(X[1],Y[1])=(2,300)
(X[2],Y[2])=(3,400)
(X[3],Y[3])=(4,500)
(X[4],Y[4])=(5,600)
```

In [47]:
```
import matplotlib.pyplot as plt
```

In [48]:
```
#SCATTER PLOT
plt.scatter(X,Y,marker="x",c='r',label="input")
plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.legend()
plt.show()
```



In [49]:
```
# Linear regression model => y=wx+b
#def linear_regression_model(X,w,b):
 #   pred_y=np.zeros(len(X))
```

```
  #  for i in range (len(X)):
   #     pred_y[i]=w*X[i]+b
    #return pred_y
```

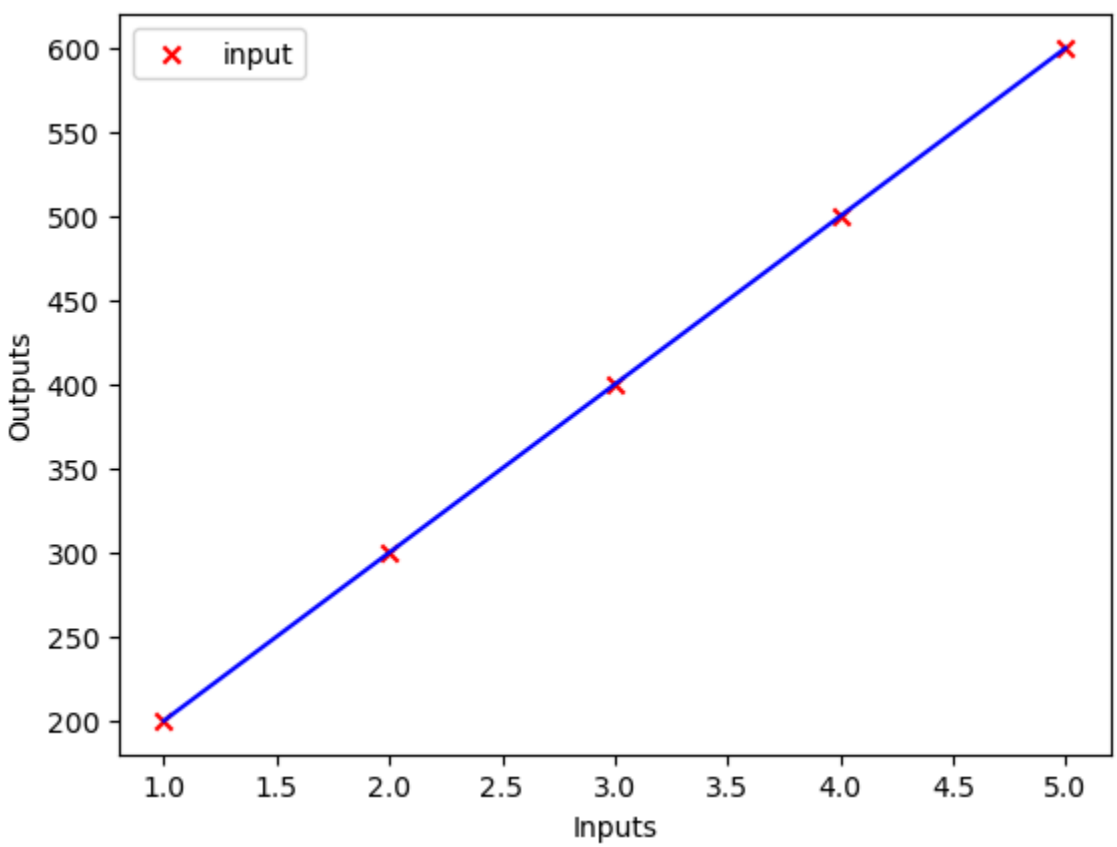
In [50]:
```
w=100
b=100
```

In [51]:
```
# Linear regression model => y=wx+b
def linear_regression_model(X,w,b):
    pred_y=np.zeros(len(X))
    for i in range (len(X)):
        pred_y[i]=w*X[i]+b
    return pred_y
```

In [52]:
```
pred_y=linear_regression_model(X,w,b)
print(pred_y)
```

```
[200. 300. 400. 500. 600.]
```

In [53]:
```
# Plot Linear Regression
plt.scatter(X,Y,marker="x",c='r',label="input")
plt.plot(X,pred_y,c='b')
plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.legend()
plt.show()
```



COST FUNCTION

In [71]:
```
def cost_function_linear_regression(pred_y,Y):
    sum_cost=0
    for i in range(len(X)):
       # pred_y=w*X[i]+b
        sum_cost=sum_cost+(pred_y[i]-Y[i])**2
    cost_total=(1/2*len(X))*sum_cost
    return cost_total
```

In [72]:
```
cost=cost_function_linear_regression(pred_y,Y)
print(cost)
```

```
0.0
```

In [ ]:

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df=pd.read_csv("Housing.csv")
```

In [3]:
```
df.head()
```

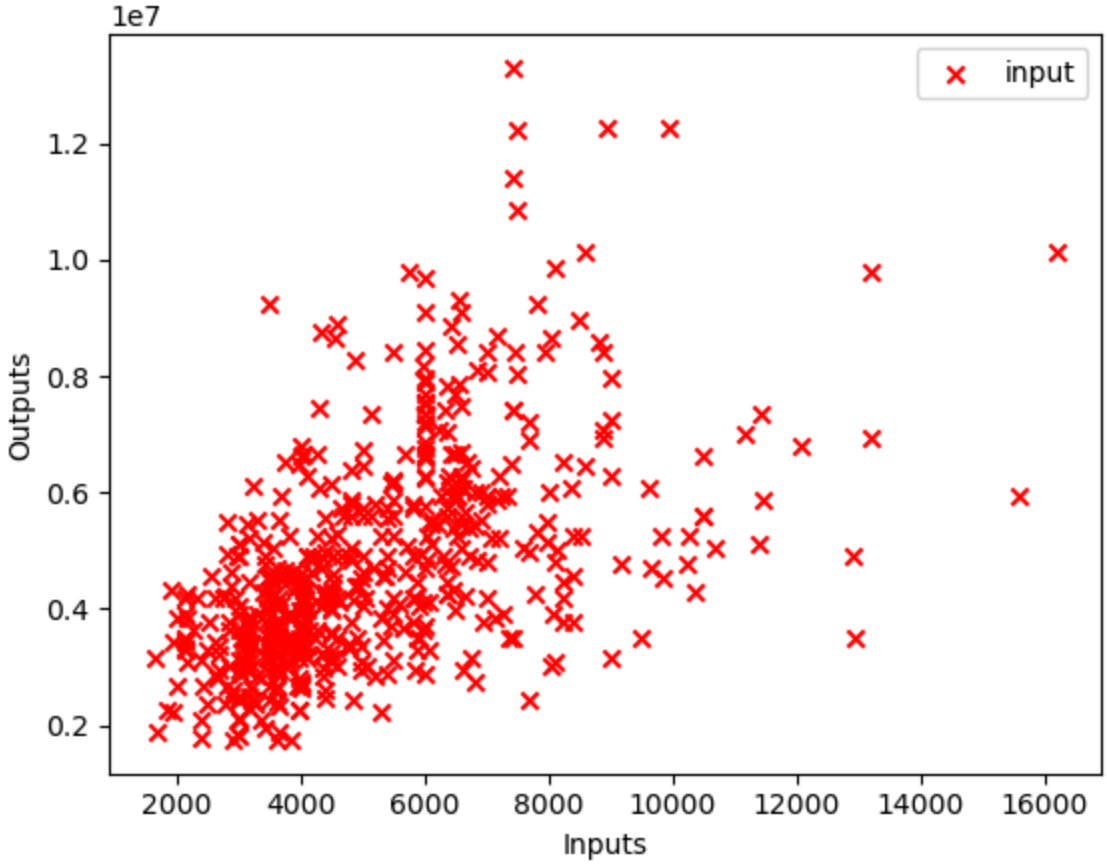
Out[3]:

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basem
0	13300000	7420	4	2	3	yes	no	
1	12250000	8960	4	4	4	yes	no	
2	12250000	9960	3	2	2	yes	no	
3	12215000	7500	4	2	2	yes	no	
4	11410000	7420	4	1	2	yes	yes	

In [4]:
```
X=df['area']
Y=df['price']
```

In [5]:
```
import matplotlib.pyplot as plt
```

In [6]:
```
#SCATTER PLOT
plt.scatter(X,Y,marker="x",c='r',label="input")
plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.legend()
plt.show()
```



In [7]:
```
# Linear regression model => y=wx+b
#def linear_regression_model(X,w,b):
 #   pred_y=np.zeros(len(X))
  #  for i in range (len(X)):
   #     pred_y[i]=w*X[i]+b
    #return pred_y
```

In [8]:
```
w=1200
b=10
```

In [9]:
```
# Linear regression model => y=wx+b
def linear_regression_model(X,w,b):
    pred_y=np.zeros(len(X))
    for i in range (len(X)):
        pred_y[i]=w*X[i]+b
    return pred_y
```

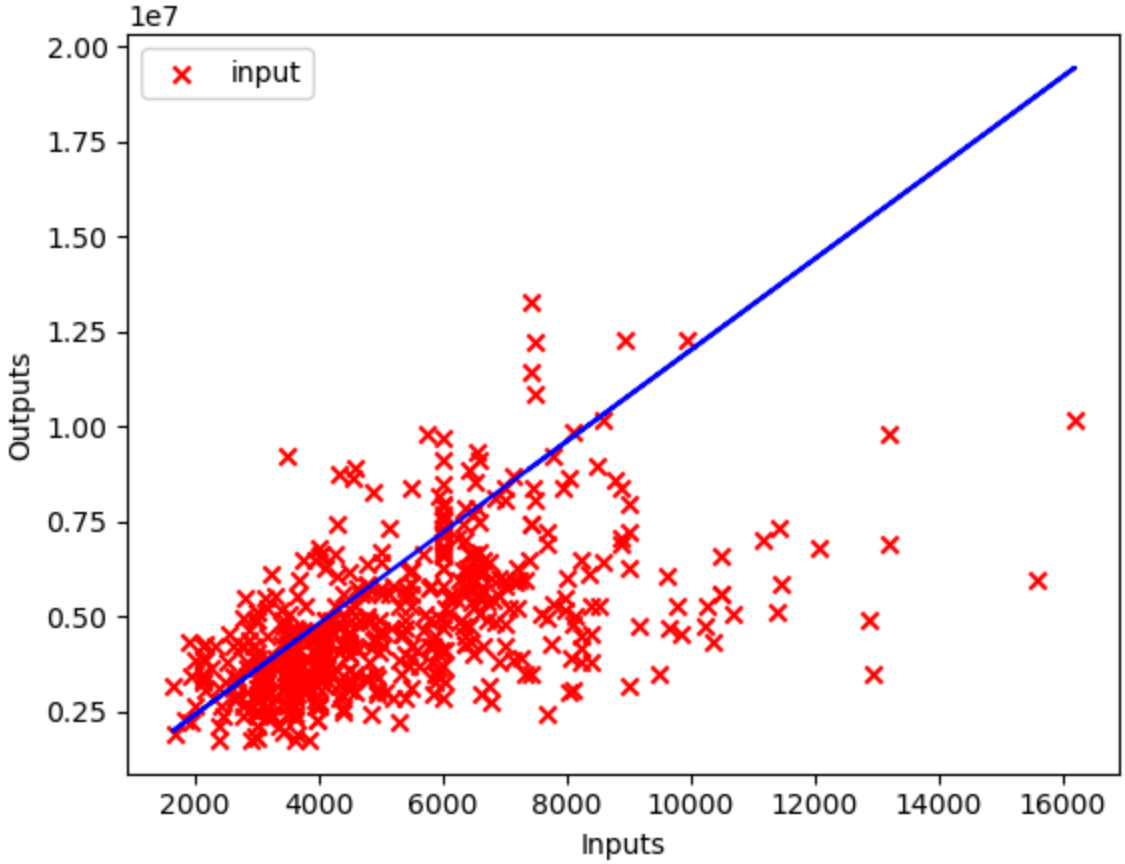
In [10]:
```
pred_y=linear_regression_model(X,w,b)
print(pred_y)
```

```
[ 8904010. 10752010. 11952010.  9000010.  8904010.  9000010. 10296010.
 19440010.  9720010.  6900010. 15840010.  7200010.  7860010.  4200010.
  9360010.  7200010.  7920010. 10200010.  5520010.  7704010.  5184010.
  8586010.  9660010.  5472010. 10560010.  7848010.  7200010. 10650010.
  9540010.  6600010.  8970010.  8400010.  5856010.  7152010.  8208010.
  8400010.  8978410. 10800010.  7200010.  7200010.  7860010.  7632010.
  7776010.  7200010.  7200010.  7200010.  7200010.  7920010.  5160010.
  8928010.  8928010.  7590010.  7200010.  6180010.  7200010.  7200010.
 13728010. 10800010.  9216010.  7200010.  7200010. 10656010.  7488010.
  7632010. 13410010. 10656010. 15840010.  9240010.  7200010. 14508010.
  4800010.  7200010.  6024010.  7920010.  4848010.  5112010.  7704010.
  7800010.  6840010.  7200010.  7200010.  4800010. 12600010.  7200010.
  4512010.  9900010.  8004010.  4752010.  8892010. 10296010.  6000010.
  8100010.  5760010.  8640010.  7200010.  4920010. 10800010.  7680010.
  7920010.  7200010.  7920010.  6600010.  6600010.  7620010.  6600010.
  5400010.  6540010.  7704010.  3888010.  7938010.  7920010. 10046410.
  5160010. 11544010.  8160010.  9600010.  8280010.  4440010.  7704010.
  8424010.  7848010.  8677210.  7504810.  8784010.  7830010. 18720010.
  8592010.  7800010.  6600010. 13752010.  5760010.  6993610.  6240010.
  5760010.  8400010.  7200010.  6480010.  5568010.  6000010.  7632010.
  6960010.  7992010. 12600010.  5760010.  5640010.  6000010. 12600010.
  6600010.  7632010.  7920010.  6163210.  5280010.  6480010.  3960010.
  4380010.  7320010.  8280010.  3380410.  9576010.  3780010.  7452010.
  7320010.  7920010.  8190010.  8052010.  7740010.  9360010.  5520010.
  5112010.  7848010.  6600010. 12322810. 10080010.  6360010.  4560010.
 11760010. 10224010.  7260010.  8502010.  3816010.  5400010.  8640010.
  4092010.  9576010.  3600010.  3600010. 13692010.  7320010.  6864010.
  4248010.  9120010. 12840010.  7920010.  5760010.  9780010.  5292010.
  9223210.  3360010.  7137610.  5040010.  5424010.  4914010.  4944010.
  6480010.  5724010.  7560010.  6960010.  3600010.  3564010.  8064010.
  5575210. 15480010.  4104010.  5994010.  5220010.  4992010.  7248010.
  8234410.  5778010.  8400010.  9720010.  4104010. 10999210.  7585210.
 12288010.  7728010.  6204010.  7200010.  4356010. 11600410.  6480010.
  5184010.  4494010.  4992010.  4656010.  6816010.  3444010.  6012010.
  5412010.  4800010.  4608010.  4512010.  4368010.  3060010.  6384010.
  6432010.  4224010. 10080010.  4920010.  5988010.  4212010.  4140010.
 11832010.  4224010.  5412010.  7062010.  4800010.  9900010.  4848010.
  7632010.  3794410.  4212010.  4500010.  4761610.  5880010.  3456010.
  5856010.  5904010.  5940010.  4680010.  5400010.  2286010.  4890010.
  4200010.  7740010.  4838410.  5280010. 12432010.  4080010.  7632010.
  7632010.  5400010.  2610010.  5232010.  9324010.  7980010.  3344410.
  6600010.  6048010.  7020010.  3132010.  3543610.  3296410.  5292010.
  4800010.  2790010.  5520010.  4368010.  6960010.  8400010.  4894810.
  4224010.  2574010.  5400010.  9900010.  4140010.  5808010.  4896010.
  4855210.  5558410.  7182010.  7272010.  4320010.  4416010.  4848010.
  6720010.  7080010.  5990410.  5208010.  3600010.  5184010.  4356010.
  4152010.  6480010.  5400010.  4152010.  4920010.  7776010.  5400010.
  4752010.  4860010.  8712010.  6600010.  3600010.  3948010.  4579210.
  9696010.  2574010.  4536010.  3816010.  6360010.  3816010.  8582410.
  4896010.  4620010.  2418010.  2611210.  4020010.  3780010.  5784010.
  4104010.  4320010.  6996010.  3427210. 10080010.  9900010.  3024010.
  8316010.  4176010.  4320010.  4848010.  7224010.  4860010.  4300810.
  3744010.  6540010.  4356010.  4356010.  6768010.  4320010.  5136010.
  4284010.  3816010.  3600010.  4224010.  7152010.  4956010.  3420010.
```

```
2730010.  4224010.  5400010.  4800010.  3780010.  5400010.  5400010.
4368010.  4620010.  5088010.  4380010.  5520010.  2562010.  3643210.
4788010.  8908810.  4176010.  4320010.  4368010.  7080010.  3744010.
8820010.  4214410. 11400010.  7056010. 15532810.  5880010.  3672010.
6384010.  2574010.  4800010.  3822010.  4620010.  2574010.  3132010.
2340010.  4848010.  5742010.  4140010.  4368010.  4200010.  5952010.
4944010.  5700010.  4464010.  4500010.  3720010.  3822010.  3240010.
2574010.  4848010.  5730010.  3000010.  3816010.  7272010.  4176010.
4550410.  4848010.  2574010.  7056010.  5400010.  4716010.  4368010.
5244010.  3220810.  5184010.  3744010.  4140010.  4783210.  4200010.
4914010.  1980010.  4140010.  8100010. 10800010.  3682810.  5400010.
6594010.  2877610.  3600010.  4620010.  4200010.  9720010.  5952010.
2592010.  3708010.  5400010.  4560010.  3708010.  3888010.  3402010.
5520010.  6091210.  4500010.  4356010.  9660010.  5222410.  3600010.
7020010.  5952010.  4320010.  4392010.  4176010.  3240010.  3780010.
7938010.  3648010.  4356010.  7200010.  6480010.  6240010.  3960010.
5220010.  3168010.  3180010.  4752010.  8160010.  4800010.  4800010.
4720810.  2400010.  4356010.  3360010.  2916010.  4176010.  4800010.
3822010.  4800010.  3492010.  4320010.  5280010.  4320010.  3456010.
3816010.  3600010.  5280010.  3600010.  3852010.  3888010.  3600010.
4200010.  5808010.  9240010.  4362010.  2970010.  3344410.  3916810.
4368010.  3816010.  2203210.  4764010.  4764010.  2340010.  6360010.
3600010.  2880010.  3600010.  4032010.  4104010.  2040010.  4378810.
3588010.  3600010.  2880010.  4344010.  3492010.  4620010.]
```

In [11]:

```
# Plot Linear Regression
plt.scatter(X,Y,marker="x",c='r',label="input")
plt.plot(X,pred_y,c='b')
plt.xlabel("Inputs")
plt.ylabel("Outputs")
plt.legend()
plt.show()
```



COST FUNCTION

In [12]:
```
def cost_function_linear_regression(pred_y,Y):
    sum_cost=0
    for i in range(len(X)):
       # pred_y=w*X[i]+b
        sum_cost=sum_cost+(pred_y[i]-Y[i])**2
    cost_total=(1/2*len(X))*sum_cost
    return cost_total
```

In [13]:
```
cost=cost_function_linear_regression(pred_y,Y)
print(cost)
```

```
1.0467956316059218e+18
```

In [ ]:

In [ ]:

In [16]:
```
import pandas as pd
```

In [17]:
```
input_data=pd.read_csv("similarity.csv")
```

In [18]:
```
input_data
```

Out[18]:

	INPUT 1	INPUT 2	INPUT 3	INPUT 4	INPUT 5
0	0	34	6	87	1
1	9	54	7	12	3
2	5	78	4	23	5
3	3	27	9	65	7
4	4	94	3	31	9

In [19]:
```
l1=input=input_data.values
```

In [20]:
```
l1
```

Out[20]:
```
array([[ 0, 34,  6, 87,  1],
       [ 9, 54,  7, 12,  3],
       [ 5, 78,  4, 23,  5],
       [ 3, 27,  9, 65,  7],
       [ 4, 94,  3, 31,  9]])
```

In [21]:
```
len(l1)
```

Out[21]: 5

EUCLIDEAN DISTANCE

In [22]:
```
import numpy as np
```

In [23]:
```
def euclidean_distance_function(x, y):
    return np.sqrt(np.sum((x - y) ** 2))
```

In [24]:
```
for i in range(len(l1)):
    for j in range(i+1,len(l1)):
        print("Euclidean Distance",euclidean_distance_function(l1[i],l1[j]))
```

```
Euclidean Distance 78.1728853247723
Euclidean Distance 77.95511529078769
Euclidean Distance 24.228082879171435
Euclidean Distance 82.61355820929153
Euclidean Distance 26.94438717061496
Euclidean Distance 59.94997914928745
Euclidean Distance 45.14421336118285
Euclidean Distance 66.31741852635702
Euclidean Distance 18.384776310850235
Euclidean Distance 75.40557008603542
```

MANHATTAN DISTANCE

In [25]:
```
def manhattan_distance(x, y):
    return np.sum(np.abs(x - y))
```

In [27]:
```
for i in range(len(l1)):
    for j in range(i+1,len(l1)):
        print("Manhattan Distance:",manhattan_distance(l1[i],l1[j]))
```

```
Manhattan Distance: 107
Manhattan Distance: 119
Manhattan Distance: 41
Manhattan Distance: 131
Manhattan Distance: 44
Manhattan Distance: 92
Manhattan Distance: 74
Manhattan Distance: 102
Manhattan Distance: 30
Manhattan Distance: 110
```

COSINE FUNCTIONS

In [33]:
```
def cosine_distance_case1(x, y):
    return np.dot(x,y)/(np.linalg.norm(x)*np.linalg.norm(y))
```

In [34]:
```
for i in range(len(l1)):
    for j in range(i+1,len(l1)):
        print("Cosine Distance:",cosine_distance_case1(l1[i],l1[j]))
```

```
Cosine Distance: 0.5524807186911643
Cosine Distance: 0.612031906209883
Cosine Distance: 0.9930863320096954
Cosine Distance: 0.6355309316709078
Cosine Distance: 0.9899725987390164
Cosine Distance: 0.5819545373864007
Cosine Distance: 0.982860002640729
Cosine Distance: 0.6321651898956506
Cosine Distance: 0.9986743001402363
Cosine Distance: 0.6554649263044877
```

In [45]:
```
from sklearn.metrics.pairwise import cosine_similarity
```

In [46]:
```
cosine_similarity(input_data)
```

Out[46]:
```
array([[1.        , 0.55248072, 0.61203191, 0.99308633, 0.63553093],
       [0.55248072, 1.        , 0.9899726 , 0.58195454, 0.98286   ],
       [0.61203191, 0.9899726 , 1.        , 0.63216519, 0.9986743 ],
       [0.99308633, 0.58195454, 0.63216519, 1.        , 0.65546493],
       [0.63553093, 0.98286   , 0.9986743 , 0.65546493, 1.        ]])
```

Minkowski Distance

In [31]:
```
def minkowski_distance(x,y,p):
```

```
    return (np.sum(np.abs(x-y)**p))**(1/p)
```

In [32]:
```
for i in range(len(l1)):
    for j in range(i+1,len(l1)):
        print("Minkowski Distance:",minkowski_distance(l1[i],l1[j],p=1))
```

```
Minkowski Distance: 107.0
Minkowski Distance: 119.0
Minkowski Distance: 41.0
Minkowski Distance: 131.0
Minkowski Distance: 44.0
Minkowski Distance: 92.0
Minkowski Distance: 74.0
Minkowski Distance: 102.0
Minkowski Distance: 30.0
Minkowski Distance: 110.0
```

In [ ]: